

CHAPTER 23

Ensemble Methods

Ensemble learning is a technique that combines the output of multiple classifiers also called weak learners to build a more robust prediction model. Ensemble methods work by combining a group of classifiers (or models) to get an enhanced prediction accuracy. The idea behind an “ensemble” is that the performance from the average of a group of classifiers will be better than each classifier on its own. So each classifier is called a “weak” learner.

Ensemble learners are usually high-performing algorithms for both classification and regression tasks and are mostly competition-winning algorithms. Examples of ensemble learning algorithms are Random Forest (RF) and Stochastic Gradient Boosting (SGB). We will motivate our discussion of ensemble methods by first discussing decision trees because ensemble classifiers such as RF and SGB are built by combining several decision tree classifiers.

Decision Trees

Decision trees, more popularly known as classification and regression trees (CART), can be visualized as a graph or flowchart of decisions. A branch connects the nodes in the graph, the last node of the graph is called a terminal node, and the topmost node is called the root. As seen in Figure 23-1, when constructing a decision tree, the root is at the top, while the branches connect nodes at lower layers until the terminal node.

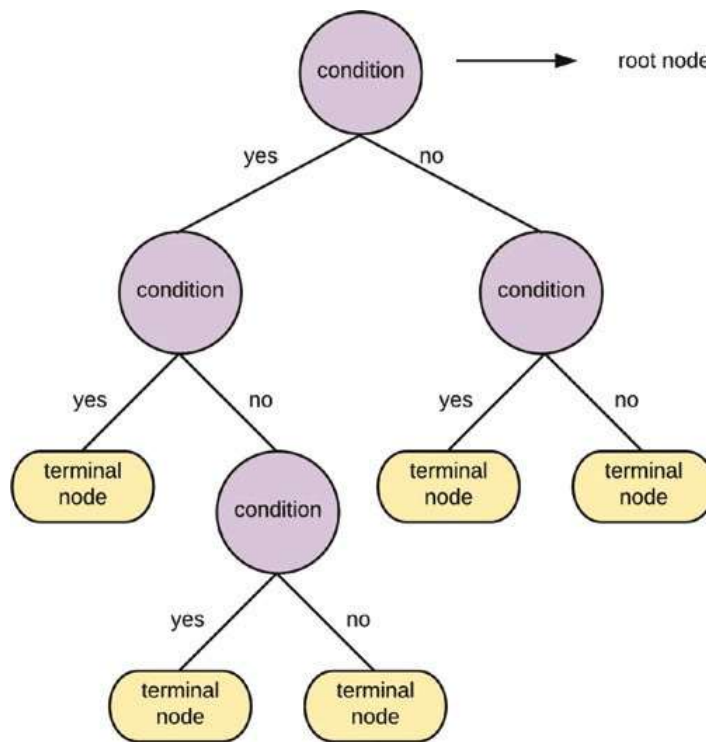


Figure 23-1. *Illustration of a decision tree*

On Regression and Classification with CART

A classification or regression tree is built by randomly splitting the set of attributes of the given dataset into distinct regions. The data points that fall within a particular region are used to form the predictor from the means of the targets in the regression case and the highest occurring class in the classification setting.

Thus, if an unseen observation or test data falls within a region, the mean or modal class is used to predict the output for regression and classification problems, respectively. In regression trees, the output variable is continuous, whereas in classification trees, the output variable is categorical. The terminal node of a regression tree takes the average of the samples in that region, while the terminal node of a classification tree is the highest occurring class in that area.

The process of splitting the features of the dataset into regions is by a greedy algorithm called recursive binary splitting. This strategy works by continuously dividing the feature space into two new branches or regions until a stopping criterion is reached.

Growing a Regression Tree

In regression trees, the recursive binary splitting technique is used to divide a particular feature in the dataset into two regions. The splitting is carried out by choosing a value of the feature that minimizes the regression error measure. This step is done for all the predictors in the dataset by finding a value that reduces the squared error of the final tree. This process is repeated continuously for every sub-tree or sub-region until a stopping criterion is reached. For example, we can stop the algorithm when no region contains less than ten observations. An example of a tree resulting from the splitting of a feature space into six regions is shown in Figure 23-2.

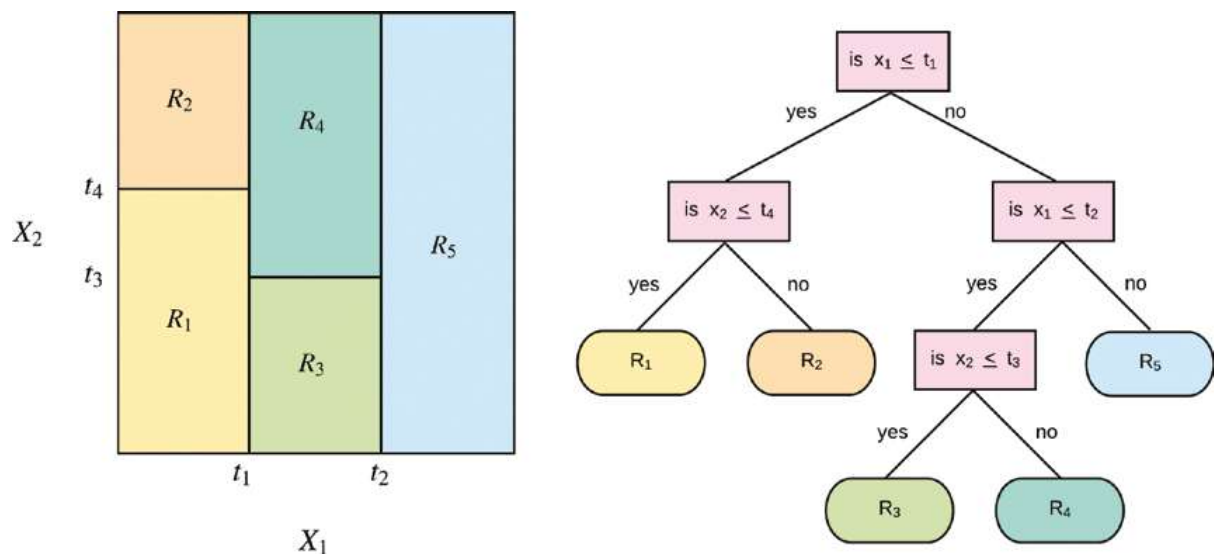


Figure 23-2. Left: An example of splitting a 2-D dataset into sub-trees/regions using the recursive binary splitting technique. Right: The resulting tree from the partitioning on the left.

Growing a Classification Tree

Growing a classification tree is very similar to the regression tree setting described in Figure 23-2. The difference here is that the error measure to minimize is no longer the squared error, but the misclassification error. This is because a classification tree is for predicting a qualitative response, where a data point is assigned to a particular region based on the modal value or the highest occurring class in that region.

Two algorithms for selecting which value to use for splitting the feature space in a classification setting are the Gini index and entropy; further discussions on these are beyond the scope of this chapter.

Tree Pruning

Tree pruning is a technique for dealing with model overfitting when growing trees. Fully grown trees have a high tendency to overfit with high variances when applied to unseen samples.

Pruning involves growing a large tree and then pruning or clipping it to create a sub-tree. By doing so, we can have a full picture of the tree performance and then select a sub-tree that results in a minimized error measure on the test dataset. The technique for selecting the best sub-tree is called the cost complexity pruning or the weakest link pruning.

Strengths and Weaknesses of CART

One of the significant advantages of CART models is that they perform well on linear and non-linear datasets. Moreover, CART models implicitly take care of feature selection and work well with high-dimensional datasets.

On the flip side, CART models can very easily overfit the dataset and fail to generalize to new examples. This downside is mitigated by aggregating a large number of decision trees in techniques like Random forests and boosting ensemble algorithms.

CART with Scikit-learn

In this section, we will implement a classification and regression decision tree classifier with Scikit-learn.

Classification Tree with Scikit-learn

In this code example, we will build a classification decision tree classifier to predict the species of flowers from the Iris dataset.

```
# import packages
from sklearn.tree import DecisionTreeClassifier
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```

# load dataset
data = datasets.load_iris()

# separate features and target
X = data.data
y = data.target

# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True)

# create the model
tree_classifier = DecisionTreeClassifier()

# fit the model on the training set
tree_classifier.fit(X_train, y_train)

# make predictions on the test set
predictions = tree_classifier.predict(X_test)

# evaluate the model performance using accuracy metric
print("Accuracy: %.2f" % accuracy_score(y_test, predictions))

'Output':
Accuracy: 0.97

```

Regression Tree with Scikit-learn

In this code example, we will build a regression decision tree classifier to predict house prices from the Boston house-prices dataset.

```

# import packages
from sklearn.tree import DecisionTreeRegressor
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from math import sqrt

```

```
# load dataset
data = datasets.load_boston()

# separate features and target
X = data.data
y = data.target

# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True)

# create the model
tree_reg = DecisionTreeRegressor()

# fit the model on the training set
tree_reg.fit(X_train, y_train)

# make predictions on the test set
predictions = tree_reg.predict(X_test)

# evaluate the model performance using the root mean square error metric
print("Root mean squared error: %.2f" % sqrt(mean_squared_error(y_test,
predictions)))

'Output':
Root mean squared error: 4.93
```

Random Forests

Random forest is a robust machine learning algorithm and is often the algorithm of choice for many classification and regression problems. It is a popular algorithm in machine learning competitions.

Random forest builds an ensemble classifier from a combination of several decision tree classifiers. This does an excellent job of reducing the variance that may be found in a single decision tree classifier.

Random forest is an improvement on the bagging ensemble algorithm (also known as bootstrap aggregation) which involves creating a large number of fully grown decision trees by repeatedly selecting random samples from the training dataset (also called bootstrapping). The result of these trees is then averaged to smoothen out the variance.

Random forest improves this bagging procedure by using only a subset of the features or attributes in the training dataset on each tree split. In doing this, Random forest creates trees whose average is more robust and less prone to high variances.

Observe that the principal distinction between bagging and Random forests is the choice of features when splitting the feature space or when building the tree. Bagging makes use of the entire features in the dataset, whereas Random forest imposes a constraint on the number of features and uses only a subset of features on each tree split to reduce the correlation of each sub-tree. Empirically, the size of features for each tree split using Random forests is the square root of the original number of predictors.

Making Predictions with Random Forests

In order to make a prediction using Random forest, the test example is passed through each trained decision tree. For the regression case, a prediction is made for a new example by taking the average of the outputs of the different trees. In the case of classification problems, the prediction is the class with the most votes from all other trees in the forest. This is best illustrated in Figure [23-3](#).

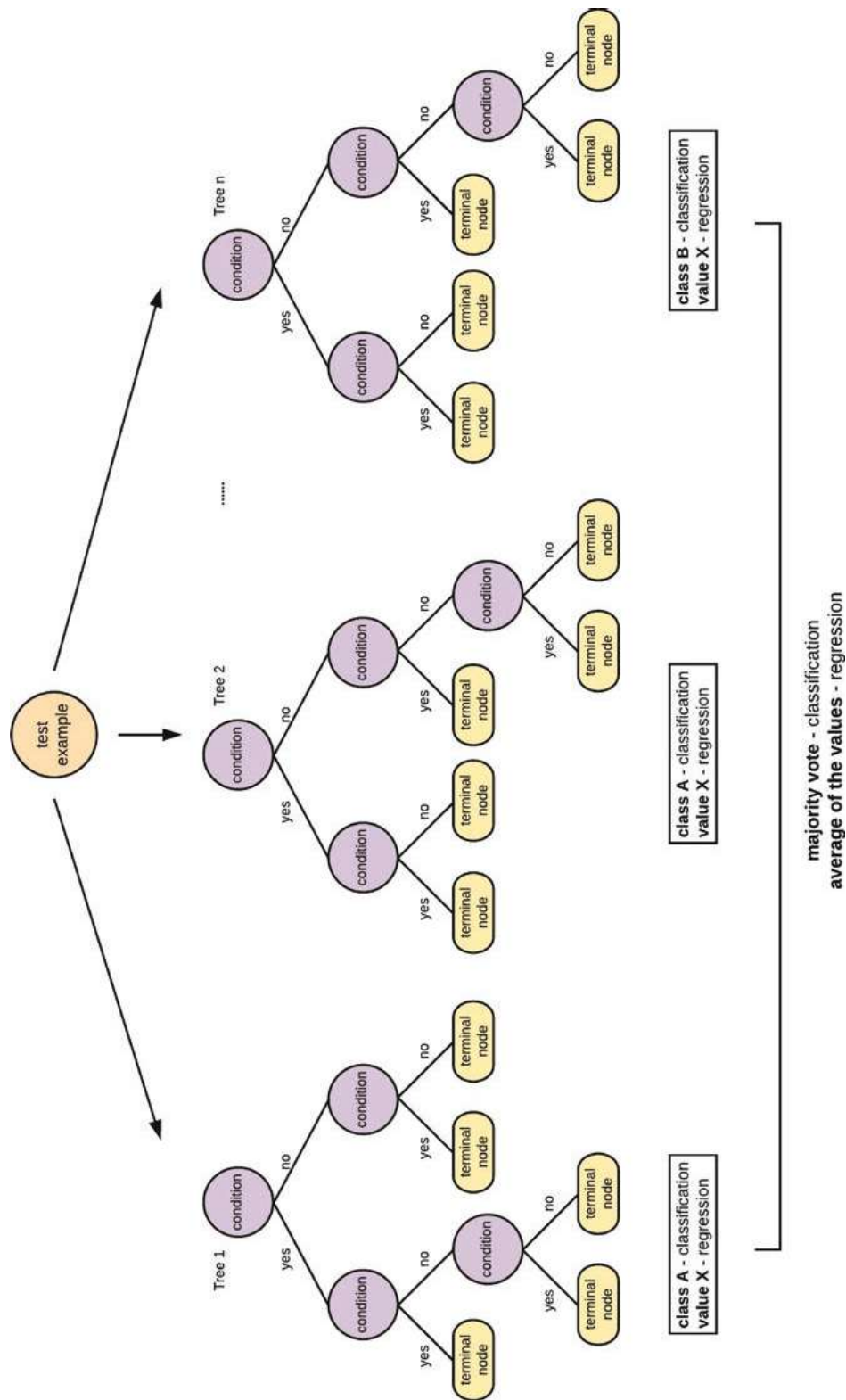


Figure 23-3. Take a majority vote to determine the final class in the classification case and the average of the values in each tree to determine the predicted value in the regression case

Random Forests with Scikit-learn

This section will implement Random forests with Scikit-learn for both regression and classification use cases.

Random Forests for Classification

In this code example, we will build a Random forest classification model to predict the species of flowers from the Iris dataset.

```
# import packages
from sklearn.ensemble import RandomForestClassifier
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# load dataset
data = datasets.load_iris()

# separate features and target
X = data.data
y = data.target

# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True)

# create the model
rf_classifier = RandomForestClassifier()

# fit the model on the training set
rf_classifier.fit(X_train, y_train)
```

```
# make predictions on the test set
predictions = rf_classifier.predict(X_test)

# evaluate the model performance using accuracy metric
print("Accuracy: %.2f" % accuracy_score(y_test, predictions))

'Output':
Accuracy: 1.00
```

Random Forests for Regression

In this code example, we will build a Random forest regression model to predict house prices from the Boston house-prices dataset.

```
# import packages
from sklearn.ensemble import RandomForestRegressor
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from math import sqrt

# load dataset
data = datasets.load_boston()

# separate features and target
X = data.data
y = data.target

# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True)
```

```
# create the model
rf_reg = RandomForestRegressor()

# fit the model on the training set
rf_reg.fit(X_train, y_train)

# make predictions on the test set
predictions = rf_reg.predict(X_test)

# evaluate the model performance using the root mean square error metric
print("Root mean squared error: %.2f" % sqrt(mean_squared_error(y_test,
predictions)))

'Output':
Root mean squared error: 2.96
```

Stochastic Gradient Boosting (SGB)

Boosting involves growing trees in succession using knowledge from the residuals of the previously grown tree. In this case, each successive tree works to improve the model of the previous tree by boosting the areas in which the previous tree did not perform so well without affecting the areas of high performance. By doing this, we iteratively create a model that reduces the residual variance when generalizing to test examples. Boosting is illustrated in Figure [23-4](#).

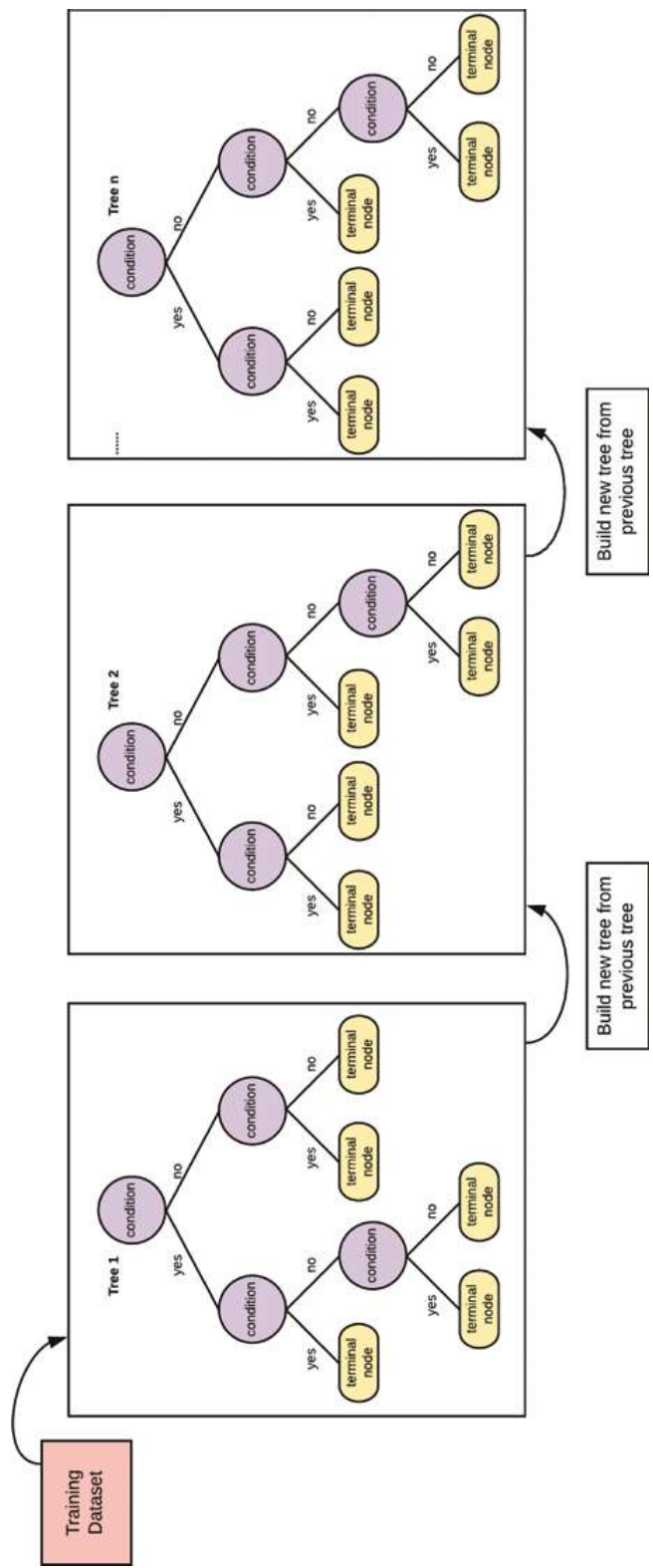


Figure 23-4. An illustration of boosting

Gradient boosting evaluates the difference of the residuals for each tree and then uses that information to determine how to split the feature space in the successive tree.

Gradient boosting employs a pseudo-gradient in computing the residuals. This gradient is the direction of quickest improvement to the loss function. The residual variance is minimized as the gradient moves in the direction of steepest descent. This movement is the same as the stochastic gradient descent algorithm discussed in Chapter 16.

Tree Depth/Number of Trees

Gradient boosting can be controlled by choosing the tree depth as a hyper-parameter to the model. In practice, a tree depth of 1 performs well, as each tree consists of just a single split. Also, the number of trees can affect the model accuracy, because gradient boosting can overfit if the number of successive trees is vast.

Shrinkage

The shrinkage hyper-parameter λ controls the learning rate of the gradient boosting model. An arbitrarily small value of λ may necessitate a larger number of trees to obtain a good model performance. However, with a small shrinkage size and tree depth $d = 1$, the residuals slowly improve by creating more varied trees to improve the worst performing areas of the model. Rule of thumb: shrinkage size is 0.01 or 0.001.

Stochastic Gradient Boosting with Scikit-learn

This section will implement SGB with Scikit-learn for both regression and classification use cases.

SGB for Classification

In this code example, we will build a SGB classification model to predict the species of flowers from the Iris dataset.

```
# import packages
from sklearn.ensemble import GradientBoostingClassifier
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# load dataset
data = datasets.load_iris()

# separate features and target
X = data.data
y = data.target

# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True)

# create the model
sgb_classifier = GradientBoostingClassifier()

# fit the model on the training set
sgb_classifier.fit(X_train, y_train)

# make predictions on the test set
predictions = sgb_classifier.predict(X_test)

# evaluate the model performance using accuracy metric
print("Accuracy: %.2f" % accuracy_score(y_test, predictions))

'Output':
Accuracy: 0.92
```

SGB for Regression

In this code example, we will build a SGB regression model to predict house prices from the Boston house-prices dataset.

```
# import packages
from sklearn.ensemble import GradientBoostingRegressor
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from math import sqrt

# load dataset
data = datasets.load_boston()

# separate features and target
X = data.data
y = data.target

# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True)

# create the model
sgb_reg = GradientBoostingRegressor ()

# fit the model on the training set
sgb_reg.fit(X_train, y_train)

# make predictions on the test set
predictions = sgb_reg.predict(X_test)

# evaluate the model performance using the root mean square error metric
print("Root mean squared error: %.2f" % sqrt(mean_squared_error(y_test,
predictions)))

'Output':
Root mean squared error: 2.86
```

XGBoost (Extreme Gradient Boosting)

XGBoost which is short for Extreme Gradient Boosting makes a couple of computational and algorithmic modifications to the stochastic gradient boosting algorithm. This enhanced algorithm is a favorite in machine learning practice due to its speed and has been the winning algorithm in many machine learning competitions. Let's go through some of the modifications made by the XGBoost algorithm.

1. **Parallel training:** XGBoost supports parallel training over multiple cores. This has made XGBoost extremely fast compared to other machine learning algorithms.
2. **Out of core computation:** XGBoost facilitates training from data not loaded into memory. This feature is a huge advantage when you're dealing with large datasets that may not necessarily fit into the RAM of the computer.
3. **Sparse data optimization:** XGBoost is optimized to handle and speed up computation with sparse matrices. Sparse matrices contain lots of zeros in its cells.

XGBoost with Scikit-learn

This section will implement XGBoost with Scikit-learn for both regression and classification use cases.

XGBoost for Classification

In this code example, we will build a XGBoost classification model to predict the species of flowers from the Iris dataset.

```
# import packages
from xgboost import XGBClassifier
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# load dataset
data = datasets.load_iris()
```

```

# separate features and target
X = data.data
y = data.target

# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True)

# create the model
xgboost_classifier = XGBClassifier()

# fit the model on the training set
xgboost_classifier.fit(X_train, y_train)

# make predictions on the test set
predictions = xgboost_classifier.predict(X_test)

# evaluate the model performance using accuracy metric
print("Accuracy: %.2f" % accuracy_score(y_test, predictions))

'Output':
Accuracy: 0.95

```

XGBoost for Regression

In this code example, we will build a XGBoost regression model to predict house prices from the Boston house-prices dataset.

```

# import packages
from xgboost import XGBRegressor
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from math import sqrt

# load dataset
data = datasets.load_boston()

# separate features and target
X = data.data
y = data.target

```

```
# split in train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True)

# create the model
xgboost_reg = XGBRegressor()

# fit the model on the training set
xgboost_reg.fit(X_train, y_train)

# make predictions on the test set
predictions = xgboost_reg.predict(X_test)

# evaluate the model performance using the root mean square error metric
print("Root mean squared error: %.2f" % sqrt(mean_squared_error(y_test,
predictions)))

'Output':
Root mean squared error: 3.69
```

In this chapter, we surveyed and implemented ensemble machine learning algorithms that combine weak decision tree learners to create a strong classifier for learning regression and classification problems. In the next chapter, we will discuss more techniques for implementing supervised machine learning models with Scikit-learn.